Principal Component Analysis

The mathematics

Projections, variance maximisation, eigenvectors, and the autoencoder view

Soumya Banerjee • Lecture notes, advanced companion • October 2026

These notes derive the results that the introductory notes used without proof: why the first principal component is an eigenvector of the covariance matrix, and why "most variance" and "closest to the data" are the same thing. The notation follows *Mathematics for Machine Learning* (see the reading list).

LEARNING OBJECTIVES

By the end of this session you should be able to:

1. use the dot product and inner product to define length, distance, orthogonality and orthonormal bases;
2. derive the orthogonal projection of a point onto a line and onto a subspace;
3. derive PCA by maximising variance, and explain why the answer is an eigenvector of the covariance matrix;
4. interpret eigenvalues and eigenvectors geometrically;
5. show that maximising variance is the same as minimising reconstruction error, and that PCA is a linear autoencoder, which links it to the representations used in large language models (LLMs).

1 Vector basics

All vectors are column vectors in $\mathbb{R}^n$, and $\boldsymbol{x}^\top$ denotes the transpose (a row).

Dot product. The dot product of $\boldsymbol{x}$ and $\boldsymbol{y}$ is the row–column product

$$\boldsymbol{x}^\top \boldsymbol{y} = \begin{pmatrix} a & b & c & \cdots \end{pmatrix} \begin{pmatrix} p \\ q \\ r \\ \vdots \end{pmatrix} = ap + bq + cr + \cdots$$

(a $1 \times n$ matrix times an $n \times 1$ matrix gives a single number). Geometrically $\boldsymbol{x}^\top \boldsymbol{y} = \|\boldsymbol{x}\| \|\boldsymbol{y}\| \cos\theta$, where θ is the angle between the vectors.

Length and distance.

$$\|\boldsymbol{x}\| = \sqrt{\boldsymbol{x}^\top \boldsymbol{x}}, \qquad \|\boldsymbol{x} - \boldsymbol{y}\| = \sqrt{(\boldsymbol{x} - \boldsymbol{y})^\top (\boldsymbol{x} - \boldsymbol{y})}.$$

Inner product. An inner product $\langle \boldsymbol{x}, \boldsymbol{y} \rangle$ *generalises* the dot product: it is any symmetric, bilinear, positive-definite way of combining two vectors into a number. The dot product is the standard example and the one PCA uses. An inner product defines length in the same way,

$$\|\boldsymbol{x}\| = \sqrt{\langle \boldsymbol{x}, \boldsymbol{x} \rangle},$$

and two vectors are *orthogonal* when $\langle \boldsymbol{x}, \boldsymbol{y} \rangle = 0$.

Orthonormal basis. Two vectors $\boldsymbol{b}_1, \boldsymbol{b}_2$ form an **orthonormal basis** of the plane if

$$\langle \boldsymbol{b}_1, \boldsymbol{b}_2 \rangle = 0, \qquad \|\boldsymbol{b}_1\| = 1, \qquad \|\boldsymbol{b}_2\| = 1,$$

that is, they are orthogonal *and* of unit length (Figure 1). The same definition extends to any number of vectors, and it is what we want from the principal components.



(a) Orthonormal **(b)** Not orthonormal

Figure 1. (a) An orthonormal basis: perpendicular vectors of unit length. (b) A basis that is neither orthogonal nor of unit length.

2 Orthogonal projection onto a line

Let $U = \text{span}(\boldsymbol{b}) = \{\alpha \boldsymbol{b} : \alpha \in \mathbb{R}\}$ be the line through the origin in the direction $\boldsymbol{b}$. Given a point $\boldsymbol{x}$, we want the point of U that is closest to it. This is the **orthogonal projection** $\pi_U(\boldsymbol{x})$:

$$\pi_U(\boldsymbol{x}) = \arg\min_{\boldsymbol{u} \in U} \|\boldsymbol{x} - \boldsymbol{u}\|.$$

(Many texts call the coordinate along $\boldsymbol{b}$ by the letter λ. We write α instead, to keep λ for eigenvalues, which appear in Section 4 and are a different thing.)

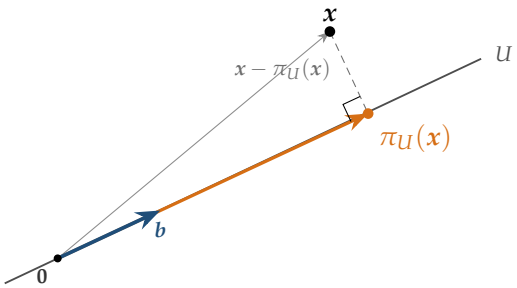


Figure 2. Orthogonal projection onto the line $U = \text{span}(\boldsymbol{b})$. $\pi_U(\boldsymbol{x})$ is a *vector* on the line, and the residual $\boldsymbol{x} - \pi_U(\boldsymbol{x})$ meets the line at a right angle.

> **KEY CONCEPT**
>
> The projection $\pi_U(\boldsymbol{x})$ is the point of U that is **closest** to $\boldsymbol{x}$. It is a vector, and it is characterised by the residual $\boldsymbol{x} - \pi_U(\boldsymbol{x})$ being **orthogonal** to U.

Derivation. Because $\pi_U(\boldsymbol{x})$ lies on U, it equals $\alpha \boldsymbol{b}$ for some α. The distance $\|\boldsymbol{x} - \pi_U(\boldsymbol{x})\|$ is smallest exactly when the residual is orthogonal to $\boldsymbol{b}$ (Figure 2):

$$\begin{aligned} \langle \boldsymbol{b}, \boldsymbol{x} - \alpha \boldsymbol{b} \rangle &= 0 \\ \langle \boldsymbol{b}, \boldsymbol{x} \rangle - \alpha \langle \boldsymbol{b}, \boldsymbol{b} \rangle &= 0 \\ \alpha &= \frac{\langle \boldsymbol{b}, \boldsymbol{x} \rangle}{\langle \boldsymbol{b}, \boldsymbol{b} \rangle}. \end{aligned}$$

For the dot product this is

$$\alpha = \frac{\boldsymbol{b}^\top \boldsymbol{x}}{\boldsymbol{b}^\top \boldsymbol{b}} = \frac{\boldsymbol{b}^\top \boldsymbol{x}}{\|\boldsymbol{b}\|^2}.$$

If $\boldsymbol{b}$ has unit length, as in an orthonormal basis, the denominator is 1 and

$$\boxed{\alpha = \boldsymbol{b}^\top \boldsymbol{x}, \qquad \pi_U(\boldsymbol{x}) = \alpha \boldsymbol{b} = \boldsymbol{b}^\top \boldsymbol{x}\, \boldsymbol{b} = \boldsymbol{b}\boldsymbol{b}^\top \boldsymbol{x}.}$$

So the coordinate of $\boldsymbol{x}$ along $\boldsymbol{b}$ is just an inner product, and $\boldsymbol{b}\boldsymbol{b}^\top$ is the projection matrix onto U.

Why orthogonal means closest. Take any other point $\alpha' \boldsymbol{b}$ of U. Since $\boldsymbol{x} - \alpha \boldsymbol{b}$ is orthogonal to $\boldsymbol{b}$, Pythagoras gives

$$\|\boldsymbol{x} - \alpha' \boldsymbol{b}\|^2 = \|\boldsymbol{x} - \alpha \boldsymbol{b}\|^2 + (\alpha - \alpha')^2 \|\boldsymbol{b}\|^2 \;\geq\; \|\boldsymbol{x} - \alpha \boldsymbol{b}\|^2.$$

Worked example. Let $\boldsymbol{x} = (3,1)^\top$ and $\boldsymbol{b} = (1,1)^\top/\sqrt{2}$. Then $\alpha = \boldsymbol{b}^\top \boldsymbol{x} = 4/\sqrt{2} = 2\sqrt{2} \approx 2.83$, so $\pi_U(\boldsymbol{x}) = \alpha \boldsymbol{b} = (2,2)^\top$ and the residual is $(1,-1)^\top$. Check: $\langle \boldsymbol{b}, (1,-1)^\top \rangle = 0$, and $\|\boldsymbol{x}\|^2 = 10 = \|\pi_U(\boldsymbol{x})\|^2 + \|\text{residual}\|^2 = 8 + 2$.

3 Projection onto a higher-dimensional subspace

The same idea works in higher dimensions (Figure 3). Let U be spanned by basis vectors $\boldsymbol{b}_1, \ldots, \boldsymbol{b}_M$, collected as the columns of $B = [\boldsymbol{b}_1, \ldots, \boldsymbol{b}_M]$. The projection $\pi_U(\boldsymbol{x})$ lies in U, so it must be a linear combination of the basis vectors:

$$\pi_U(\boldsymbol{x}) = \sum_{i=1}^{M} \alpha_i \boldsymbol{b}_i = B\boldsymbol{\alpha}.$$

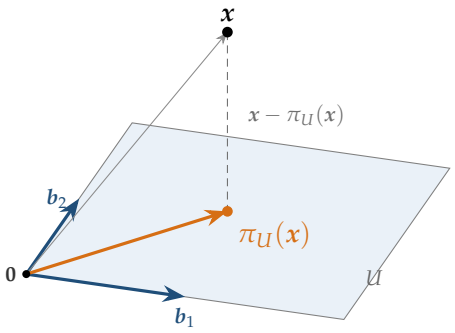


Figure 3. Projection onto a plane $U = \text{span}(\boldsymbol{b}_1, \boldsymbol{b}_2)$ in three dimensions. The residual $\boldsymbol{x} - \pi_U(\boldsymbol{x})$ is orthogonal to every vector in the plane.

Derivation. We again want the distance to be minimal, so the residual must be orthogonal to *every* basis vector:

$$\langle \boldsymbol{b}_1, \boldsymbol{x} - \pi_U(\boldsymbol{x}) \rangle = 0, \quad \langle \boldsymbol{b}_2, \boldsymbol{x} - \pi_U(\boldsymbol{x}) \rangle = 0, \quad \ldots, \quad \langle \boldsymbol{b}_M, \boldsymbol{x} - \pi_U(\boldsymbol{x}) \rangle = 0.$$

In matrix notation, with $\pi_U(\boldsymbol{x}) = B\boldsymbol{\alpha}$ and the dot product,

$$B^\top (B\boldsymbol{\alpha} - \boldsymbol{x}) = \boldsymbol{0}$$

$$B^\top B\, \boldsymbol{\alpha} = B^\top \boldsymbol{x}$$

$$\boxed{\boldsymbol{\alpha} = (B^\top B)^{-1} B^\top \boldsymbol{x}}$$

(assuming the columns of B are linearly independent, so that $B^\top B$ is invertible). Here $\boldsymbol{\alpha}$ holds the *coordinates* of the projection in the basis B; it is not an eigenvalue.

KEY CONCEPT

If the basis is **orthonormal**, then $B^\top B = I$ and the formulas collapse:

$$\boldsymbol{\alpha} = B^\top \boldsymbol{x}, \qquad \pi_U(\boldsymbol{x}) = BB^\top \boldsymbol{x}.$$

No matrix inverse is needed: each coordinate is an inner product $\alpha_i = \boldsymbol{b}_i^\top \boldsymbol{x}$. This is exactly the form PCA uses.

Building the basis one vector at a time. PCA chooses the basis *iteratively*: find the first principal component $\boldsymbol{b}_1$, then the second, $\boldsymbol{b}_2$, constrained to be orthogonal to the first, and so on, up to the dimension of the data. Section 4 shows how each vector is found.

4 PCA as variance maximisation

4.1 Setting up

Let $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N \in \mathbb{R}^D$ be data points with **mean zero** (we centre the data first; if the features are on different scales we also standardise them, as in the introductory notes). The **covariance** between two variables is the expected value of the product of their deviations from their means, so for centred data the **data covariance matrix** is

$$S = \frac{1}{N} \sum_{n=1}^{N} \boldsymbol{x}_n \boldsymbol{x}_n^\top \in \mathbb{R}^{D \times D}.$$

Its (i, j) entry is the covariance of features i and j. (Dividing by $N - 1$ instead of N changes nothing below, because it only rescales S.)

We compress each data point to a lower-dimensional code $\boldsymbol{z}_n \in \mathbb{R}^M$ with $M < D$:

$$\boldsymbol{z}_n = B^\top \boldsymbol{x}_n, \qquad B = [\boldsymbol{b}_1, \boldsymbol{b}_2, \ldots, \boldsymbol{b}_M],$$

where B is the **projection matrix** and its columns are orthonormal: $\boldsymbol{b}_i^\top \boldsymbol{b}_i = 1$ and $\boldsymbol{b}_i^\top \boldsymbol{b}_j = 0$ for $i \neq j$. By Section 3, $B\boldsymbol{z}_n = BB^\top \boldsymbol{x}_n$ is the orthogonal projection of $\boldsymbol{x}_n$ onto the subspace spanned by the columns of B.

KEY CONCEPT

PCA assumes a **linear** relationship between a data point and its lower-dimensional representation: $\boldsymbol{z}_n = B^\top \boldsymbol{x}_n$.

4.2 The first principal component

We look for the direction $\boldsymbol{b}_1$ along which the projected data have the largest variance (Figure 4). The projected coordinate of $\boldsymbol{x}_n$ is $z_{1n} = \boldsymbol{b}_1^\top \boldsymbol{x}_n$. Since the data are centred, so are the z_{1n}, and their variance is

$$V_1 = \frac{1}{N} \sum_{n=1}^{N} z_{1n}^2.$$

The dot product is symmetric, $\boldsymbol{b}_1^\top \boldsymbol{x}_n = \boldsymbol{x}_n^\top \boldsymbol{b}_1$, so

$$\begin{aligned} V_1 &= \frac{1}{N} \sum_{n=1}^{N} (\boldsymbol{b}_1^\top \boldsymbol{x}_n)(\boldsymbol{x}_n^\top \boldsymbol{b}_1) \\ &= \boldsymbol{b}_1^\top \Big(\frac{1}{N} \sum_{n=1}^{N} \boldsymbol{x}_n \boldsymbol{x}_n^\top \Big) \boldsymbol{b}_1 \qquad (\boldsymbol{b}_1 \text{ does not depend on } n) \\ &= \boldsymbol{b}_1^\top S \, \boldsymbol{b}_1. \end{aligned}$$

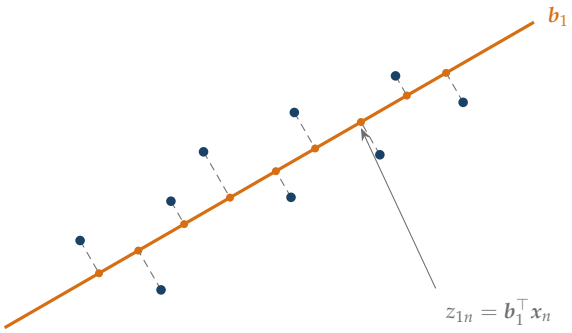


Figure 4. Projecting centred data onto a direction $\boldsymbol{b}_1$. Blue points are the data $\boldsymbol{x}_n$; orange points are the projections, whose positions along the line are the coordinates z_{1n}. PCA picks the direction that makes these as spread out as possible.

So the task is: *find* $\boldsymbol{b}_1$ *such that* $\boldsymbol{b}_1^\top S \boldsymbol{b}_1$ *is maximised*. Without a constraint this has no answer (doubling $\boldsymbol{b}_1$ quadruples the variance), so we require unit length, $\boldsymbol{b}_1^\top \boldsymbol{b}_1 = 1$. Use a Lagrange multiplier λ_1:

$$\mathcal{L}(\boldsymbol{b}_1, \lambda_1) = \boldsymbol{b}_1^\top S \boldsymbol{b}_1 + \lambda_1 \big(1 - \boldsymbol{b}_1^\top \boldsymbol{b}_1\big).$$

Setting the gradient with respect to $\boldsymbol{b}_1$ to zero (with S symmetric) gives $2S\boldsymbol{b}_1 - 2\lambda_1 \boldsymbol{b}_1 = \boldsymbol{0}$, that is

$$\boxed{S\boldsymbol{b}_1 = \lambda_1 \boldsymbol{b}_1.}$$

> **KEY CONCEPT**
>
> The direction of maximum variance must be an **eigenvector** of the data covariance matrix S. Among all eigenvectors we choose the one with the **largest eigenvalue**, because the variance it captures equals its eigenvalue (shown next).

The variance is the eigenvalue. Substituting $S\boldsymbol{b}_1 = \lambda_1 \boldsymbol{b}_1$ and using $\boldsymbol{b}_1^\top \boldsymbol{b}_1 = 1$,

$$V_1 = \boldsymbol{b}_1^\top S \boldsymbol{b}_1 = \lambda_1 \boldsymbol{b}_1^\top \boldsymbol{b}_1 = \lambda_1, \qquad \boxed{V_1 = \lambda_1.}$$

So λ_1 is the *variance* of the data along the first principal component, and $\sqrt{\lambda_1}$ is its standard deviation (this is what software such as R's `prcomp` reports as the "standard deviation" of each component).

4.3 Further components

For the second component we maximise $\boldsymbol{b}_2^\top S \boldsymbol{b}_2$ subject to $\boldsymbol{b}_2^\top \boldsymbol{b}_2 = 1$ and $\boldsymbol{b}_2^\top \boldsymbol{b}_1 = 0$. The same argument shows that $\boldsymbol{b}_2$ is the eigenvector with the second-largest eigenvalue, and so on. In summary:

1. Centre (and, if appropriate, standardise) the data.
2. Compute the covariance matrix S.
3. Find the eigenvalues $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_D \geq 0$ and orthonormal eigenvectors $\boldsymbol{b}_1, \ldots, \boldsymbol{b}_D$ of S.
4. Keep the first M eigenvectors as the columns of B; the codes are $\boldsymbol{z}_n = B^\top \boldsymbol{x}_n$.

The M components capture a total variance $\sum_{m=1}^{M} \lambda_m$ out of $\mathrm{tr}(S) = \sum_{d=1}^{D} \lambda_d$, so the proportion of variance explained is $\sum_{m \leq M} \lambda_m / \sum_d \lambda_d$. This is the quantity plotted in the scree plot of the introductory notes. The scores of different components are uncorrelated, because $B^\top S B$ is diagonal.

Loadings. In the introductory notes the loadings ϕ_{jm} were the entries of the unit-length eigenvector $\boldsymbol{b}_m$, so $B = \Phi$ and $\boldsymbol{z}_n = \Phi^\top \boldsymbol{x}_n$. Another common convention scales each eigenvector by $\sqrt{\lambda_m}$: for standardised data, $\sqrt{\lambda_m}\,\phi_{jm}$ is the correlation between feature j and component m. Always check which convention a piece of software uses. As the introductory notes stress, it is the *relative* sizes within a loading vector that carry the interpretation.

Worked example. Take $S = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$. The eigenvalues solve $(2-\lambda)^2 - 1 = 0$, so $\lambda_1 = 3$ and $\lambda_2 = 1$, with orthonormal eigenvectors

$$\boldsymbol{b}_1 = \tfrac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \qquad \boldsymbol{b}_2 = \tfrac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix}.$$

Check: $S\boldsymbol{b}_1 = \frac{1}{\sqrt{2}}(3,3)^\top = 3\boldsymbol{b}_1$. The first component captures $V_1 = 3$ of the total variance $\operatorname{tr}(S) = 4$, that is 75%. (This S is twice a correlation matrix with $\rho = 0.5$. For any standardised two-dimensional data with positive correlation, the first component lies at 45°, as in Figure 2 of the introductory notes.)

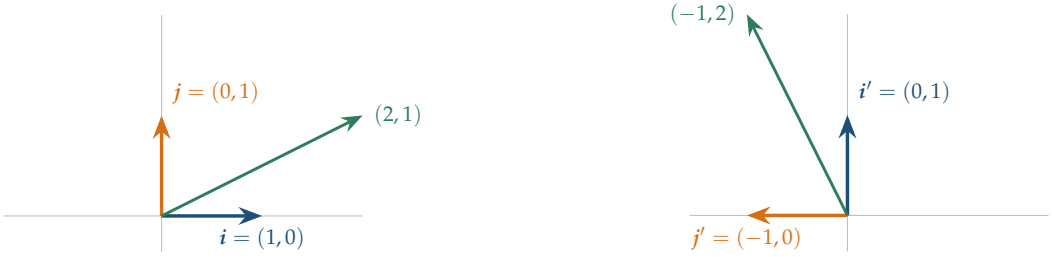
5 Eigenvectors and eigenvalues: building intuition

5.1 Matrix multiplication as a linear transformation

A matrix is a recipe for moving space around. Take the standard basis $\boldsymbol{i} = (1,0)^\top$ and $\boldsymbol{j} = (0,1)^\top$ and rotate the plane by 90° anticlockwise (Figure 5). Then $\boldsymbol{i}$ lands on $(0,1)^\top$ and $\boldsymbol{j}$ on $(-1,0)^\top$. The **columns** of the matrix are where the basis vectors land:

$$R = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}, \qquad R \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} -y \\ x \end{pmatrix}.$$

For example, the point $(2,1)^\top$ moves to $(-1,2)^\top$. Every other vector follows, because $(x,y)^\top = x\boldsymbol{i} + y\boldsymbol{j}$ and the transformation is linear.



(a) Before **(b)** After a 90° rotation

Figure 5. Matrix multiplication as a linear transformation. The columns of R record where $\boldsymbol{i}$ and $\boldsymbol{j}$ land, and every other vector, such as $(2,1)^\top$, is carried along.

5.2 Which directions survive?

> **KEY CONCEPT**
>
> For a square matrix A, an **eigenvector** is a direction that the transformation only *stretches*, never turns: $Av = \lambda v$. The **eigenvalue** λ says **how much you stretch** along that direction.

- **A pure stretch.** Let $A = \operatorname{diag}(2, \frac{1}{2})$. A vector along the horizontal axis stays on its line and doubles in length, so it is an eigenvector with $\lambda = 2$; the vertical axis is an eigenvector with $\lambda = \frac{1}{2}$. A diagonal vector, in contrast, changes direction, so it is not an eigenvector (Figure 6a).

- **A rotation in the plane.** Is there any line or vector in the plane that does not change under a 90° rotation? No: every direction is turned. Algebraically $Rv = \lambda v$ would need $\lambda^2 + 1 = 0$, which has no real solution, so R has no real eigenvectors.
- **A rotation in space.** Move outside the plane. A spinning cricket ball keeps turning, yet its *axis of rotation* does not move: that axis is an eigenvector with eigenvalue 1 (Figure 6b).



(a) Stretch by $(2, \frac{1}{2})$ **(b)** Rotation in space

Figure 6. (a) Under a stretch, the blue direction keeps its line (an eigenvector) while the orange direction is turned. Dashed grey shows the vectors before the transformation. (b) A spinning ball: the rotation axis is the one direction left unchanged.

5.3 The covariance matrix as an ellipse

The covariance matrix S is **symmetric**, and symmetric matrices have two properties that PCA relies on: their eigenvalues are real, and their eigenvectors can be chosen to be mutually orthogonal (and of unit length). So the eigenvectors of S automatically form the orthonormal basis that Section 4 requires. In addition, S is positive semi-definite, so every eigenvalue is ≥ 0, as a variance must be.

Geometrically, S describes an ellipse (Figure 7): the eigenvectors point along its axes, and the semi-axis lengths are $\sqrt{\lambda_m}$. The eigenvector with the largest eigenvalue is the long axis, which is the direction in which the data are most spread out.

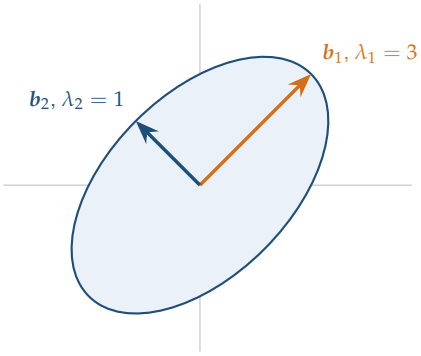
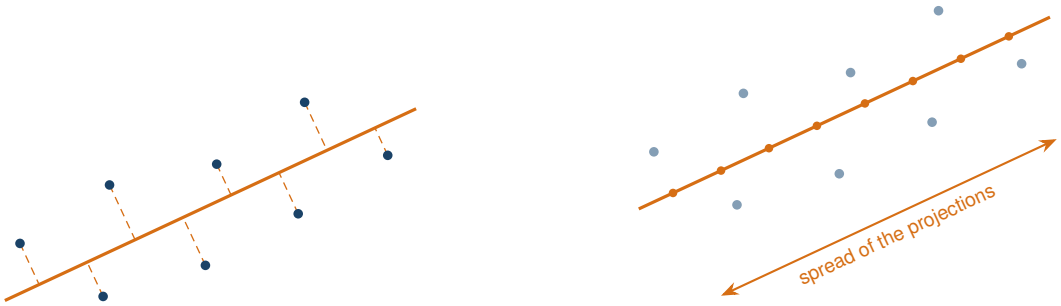


Figure 7. The covariance matrix $S = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$ drawn as an ellipse (the one-standard-deviation contour $x^\top S^{-1} x = 1$). The eigenvectors lie along its axes, with semi-axis lengths proportional to $\sqrt{\lambda}$.

> **KEY CONCEPT**
>
> **Eigenvector:** the direction of the transformation, here a principal axis of the data. **Eigenvalue:** how much you stretch along it, here the **variance** the data have in that direction.

6 Another derivation: minimising reconstruction error

So far we maximised the variance of the projections. The same principal components also come from asking for the line that makes the *residuals* as small as possible (Figure 8): minimise the distances from the data to their projections, **or** equivalently maximise the variance of the projections.



(a) Minimise these residuals **(b)** Maximise the variance

Figure 8. Two views of the same optimisation. (a) Choose the line that minimises the (squared) residuals shown by the dashed segments. (b) Choose the line that maximises the spread of the projected points.

Why they agree. Let $\tilde{x}_n = BB^\top x_n$ be the reconstruction of x_n from the code $z_n = B^\top x_n$. The residual $x_n - \tilde{x}_n$ is orthogonal to $\tilde{x}_n$ (Section 3), so by Pythagoras

$$\|x_n\|^2 = \|\tilde{x}_n\|^2 + \|x_n - \tilde{x}_n\|^2, \qquad \text{and} \qquad \|\tilde{x}_n\|^2 = \|B^\top x_n\|^2 = \|z_n\|^2.$$

Averaging over the data, with $J = \frac{1}{N}\sum_n \|x_n - \tilde{x}_n\|^2$ the mean squared reconstruction error,

$$\underbrace{\tfrac{1}{N}\textstyle\sum_n \|x_n\|^2}_{\mathrm{tr}(S)\ \text{(fixed)}} = \underbrace{\tfrac{1}{N}\textstyle\sum_n \|z_n\|^2}_{\text{variance captured}} + J.$$

The left-hand side does not depend on B, so **minimising J is the same as maximising the captured variance.** At the optimum the error is the variance we threw away:

$$J = \sum_{d=M+1}^{D} \lambda_d.$$

> **WATCH AND EXPLORE**
>
> For yet another derivation (via the singular value decomposition), see page 16 of the notes `mathematics_data_science.pdf` and the companion repository https://github.com/neelsoumya/public_teaching_unsupervised_learning.

7 Encoders, decoders, and the link to LLMs

Read the formulas of Sections 3 and 5 as a pipeline (Figure 9): a high-dimensional input x is **encoded** into a low-dimensional code z and then **decoded** back into a reconstruction $\tilde{x}$, and the **reconstruction error** (the loss) measures what was lost.

Two design choices define a method of this kind, and varying them gives the main families of unsupervised learning:

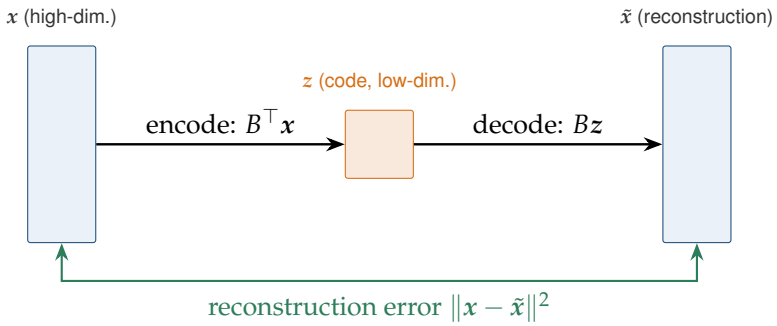


Figure 9. PCA as a linear autoencoder. The encoder is $z = B^\top x$, the decoder is $\tilde{x} = Bz$, and the loss is the mean squared reconstruction error.

	PCA	**Beyond PCA**
Distance (loss)	squared Euclidean distance, from the dot product	other inner products or distances
Encoder and decoder	linear maps $B^\top$ and B	non-linear neural networks

Why this matters for LLMs.

- **Autoencoders.** A network with linear layers, squared-error loss and a low-dimensional bottleneck learns the same subspace as PCA. Replacing the maps with non-linear networks gives general autoencoders, which can capture curved structure. Sparse autoencoders trained on LLM activations are one current tool in mechanistic interpretability.
- **Inner products everywhere.** The similarity of two embedding vectors is usually measured by the cosine $\langle x, y \rangle / (\|x\| \|y\|)$, and the attention scores inside a transformer are (scaled) dot products between query and key vectors. Sections 1 to 3 are the geometry behind both.
- **Looking inside.** Projecting high-dimensional embeddings or activations onto their first few principal components is a standard way to visualise and probe them, using exactly the encoder $z = B^\top x$ derived above.

8 Watching and further study

WATCH AND EXPLORE

- The 3Blue1Brown series *Essence of linear algebra* on YouTube, in particular the videos on linear transformations and matrices, and on eigenvectors and eigenvalues. Recommended before or alongside Section 5.
- The cricket-ball rotation demonstration (Figure 6b): which direction stays fixed as the ball spins?
- The companion repository: https://github.com/neelsoumya/public_teaching_unsupervised_learning.

SUMMARY

- The orthogonal projection $\pi_U(\boldsymbol{x})$ is the point of a subspace **closest** to $\boldsymbol{x}$; it is characterised by an **orthogonal residual**.
- With an orthonormal basis B, the coordinates are $\boldsymbol{z} = B^\top \boldsymbol{x}$ and the projection is $BB^\top \boldsymbol{x}$.
- The variance of the data along a unit vector $\boldsymbol{b}$ is $\boldsymbol{b}^\top S\boldsymbol{b}$; maximising it gives $S\boldsymbol{b} = \lambda \boldsymbol{b}$.
- The principal components are the eigenvectors of the covariance matrix, and each eigenvalue is the **variance** captured by its component.
- Maximising variance $\Leftrightarrow$ minimising reconstruction error, and the error equals the sum of the discarded eigenvalues.
- PCA is a linear autoencoder; non-linear encoders and decoders lead to the representations used in LLMs.

DISCUSSION QUESTIONS

1. Why does maximising $\boldsymbol{b}^\top S\boldsymbol{b}$ have no solution without the constraint $\boldsymbol{b}^\top \boldsymbol{b} = 1$? What does the constraint mean geometrically?
2. The rotation R has no real eigenvectors in the plane. What would the (complex) eigenvalues be, and why do covariance matrices never have this problem?
3. If you train a linear autoencoder with a two-dimensional bottleneck by minimising squared error, what will its decoder span? Will it necessarily recover $\boldsymbol{b}_1$ and $\boldsymbol{b}_2$ individually?
4. Which of the ideas in Section 1 (dot product, length, angle) does cosine similarity use?

Further reading

[1] M. P. Deisenroth, A. A. Faisal and C. S. Ong, *Mathematics for Machine Learning*, Cambridge University Press, 2020 (orthogonal projections and PCA; the notation here follows this book).

[2] G. Sanderson (3Blue1Brown), *Essence of linear algebra*, video series, YouTube.

[3] G. James, D. Witten, T. Hastie and R. Tibshirani, *An Introduction to Statistical Learning*, 2nd ed., Springer, 2021.